

Graph Contractions for Calculating Correlation Functions in Lattice QCD

Jie Chen
chen@jlab.org
Scientific Computing
Jefferson Lab
Newport News, VA, USA

Robert G. Edwards
edwards@jlab.org
Theory Center
Jefferson Lab
Newport News, VA, USA

Weizhen Mao
wm@cs.wm.edu
Department of Computer Science
William & Mary
Williamsburg, VA, USA

ABSTRACT

Computing correlation functions for many-particle systems in Lattice QCD is vital to extract nuclear physics observables like the energy spectrum of hadrons such as protons. However, this type of calculation has long been considered to be very challenging and computing-resource intensive because of the complex nature of a hadron composed of quarks with many degrees of freedom. In particular, a correlation function can be calculated through a sum of all possible pairs of quark contractions, each of which is a batched tensor contraction, dictated by *Wick's* theorem. Because the number of terms of this sum can be very large for any hadronic system of interest, fast evaluation of the sum faces several challenges: an extremely large number of contractions, a huge memory footprint at runtime, and the speed of tensor contractions. In this paper, we present a Lattice QCD analysis software suite, *Redstar*, which addresses these challenges by utilizing novel algorithmic and software engineering methods targeting modern computing platforms such as many-core CPUs and GPUs. In particular, *Redstar* represents every term in the sum of a correlation function by a graph, applies efficient graph algorithms to reduce the number of contractions to lower the cost of computations, and minimizes the total memory footprint. Moreover, *Redstar* carries out the contractions on either CPUs or GPUs utilizing an internal and highly efficient *Hadron* contraction library. Specifically, we illustrate some important algorithmic optimizations of *Redstar*, show various key design features of *Hadron* library, and present the speedup values due to the optimizations along with performance figures for calculating six correlations functions on four computing platforms.

CCS CONCEPTS

• Applied computing → Physics; • Mathematics of computing → Graph algorithms; • General and reference → Performance; Evaluation.

KEYWORDS

Redstar, graph contractions, correlation functions, LQCD, hadron contractions, tensor contractions

ACM Reference Format:

Jie Chen, Robert G. Edwards, and Weizhen Mao. 2023. Graph Contractions for Calculating Correlation Functions in Lattice QCD. In *Platform for Advanced Scientific Computing Conference (PASC '23)*, June 26–28, 2023, Davos, Switzerland. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3592979.3593409>

1 INTRODUCTION

Quantum chromodynamics (QCD) is the fundamental theory to describe the dynamics of quarks and gluons in hadrons. The gluons are gauge field interacting with quarks with a coupling that grows as the distance between quarks increases. Because of this property, the perturbation theory is not applicable when the coupling becomes too strong in the case of low energy scale arena. Lattice QCD (LQCD) [4] simulations thus have become a most powerful tool to analyze the non-perturbative nature of QCD through the process of discretizing four-dimensional spacetime onto a regular lattice where quarks live on the sites and gluons fields are defined on the links connecting neighboring sites.

Lattice QCD calculations are generally comprised of three main steps. The first step is the generation of ensembles of gauge field configurations, which is a Monte-Carlo sampling of the strong force fields in the vacuum and is carried out mostly on supercomputers. The second step of the process is the computation of quark propagators to follow the movement of quarks by releasing *color charge* of a quark in the background of the strong force (gauge) field. A quark propagator is labeled at each end by indices corresponding to the position, color, and other quantum numbers. In the third step called *analysis* on which this paper focuses, these indices will be contracted into “colorless” correlation functions to deduce the quantities of interest known as physics observables such as the energy spectrum of hadrons.

In particular, a 2-pt correlation function between creation and annihilation operator $\chi^\dagger$ and χ at Euclidean t and t' can be written as $C(t', t) = \langle \chi(t') \chi^\dagger(t) \rangle$. The operator χ for n_q quarks of nucleons can be described as a sum of tensor product of individual quark operators, taking the form

$$\chi = \sum_{(\vec{a})} W^{a_1, \dots, a_{n_q}} q(a_1) q(a_2) \dots q(a_{n_q}) \quad (1)$$

where $q(a_i)$ is a quark operator for a quark with quantum numbers represented by a_i , and W is an antisymmetric tensor that is nonzero only when none of the indices are equal and the combination of the quark quantum numbers is equal to the nuclear quantum number [5]. If there are N possible quark indices, the number of non-zero terms in the sum is written as



This work is licensed under a Creative Commons Attribution International 4.0 License.
PASC '23, June 26–28, 2023, Davos, Switzerland
© 2023 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0190-0/23/06.
<https://doi.org/10.1145/3592979.3593409>

$$\frac{N!}{n_q!(N - n_q)!} \quad (2)$$

after accounting for permutations. Therefore, the 2-pt correlation function can be written as

$$C(t', t) = \sum_{(\vec{a}, \vec{a}')} W_{a_1', \dots, a_{n_q}'}^{a_1, \dots, a_{n_q}} \langle q(a_1) \dots q(a_{n_q}) \bar{q}(a_1') \dots \bar{q}(a_{n_q}') \rangle \quad (3)$$

By defining a quark propagator $S_{i,j} := S(a_i; a_j') = q_i \bar{q}_j$, which is also called a *quark contraction*, and utilizing Wick's theorem [3, 20], the correlation function can be rewritten as

$$C(t', t) = \left(\prod_{l=1}^{n_q-1} (-1)^l \right) \sum_{\vec{k} \in S_{n_q}} \epsilon^{k_1, \dots, k_{n_q}} S_{1,k_1} \dots S_{n_q, k_{n_q}'} \quad (4)$$

where S_{n_q} is the set of the $n_q!$ permutations of $\{1, \dots, n_q\}$, and ϵ is the totally anti-symmetric tensor.

From equations (2,3,4), a correlation function can be evaluated by a sum, dubbed *Wick contractions*, consisting all possible pairs of the quark propagators for a given set of quantum numbers. The number of terms of this sum scales as factorial of the number of quark fields in a correlation function. For example, the number of the terms can reach 10,000 for a two-meson system and can top 100,000 for a multi-baryon system, where a meson contains two quarks whereas a baryon has three quarks. Furthermore, each term consists a list of quark propagators, each of which can be represented by either a list of matrix ($N \times N$) multiplications for meson systems or a set of tensor ($N \times N \times N$) contractions involving baryons, where N is in the **100s** for typical LQCD calculations. Clearly, the calculation of correlation functions is time consuming and expensive. More importantly, the total computing cost of the *analysis* step can surpass those of the steps for generating gauge fields and propagators because the calculations of many correlation functions are carried out on a large ensemble of gauge fields to increase statistics accuracy.

Therefore, it is crucial to speed up this type of calculations by taking advantages of hardware computing capabilities along with advanced algorithms and software techniques. In particular, the following algorithmic optimizations should boost the overall performance: reducing the contraction cost of a single term in the sum of *Wick contractions*; minimizing the total number of contractions from all the terms of *Wick contractions*; reducing the memory usage of the calculation to enable this type of computations for more complex and larger hadronic systems. In addition, software acceleration of tensor contractions on modern computing platforms such as GPUs should provide further performance enhancement.

In this paper, we present how the *Redstar*, which is an end-to-end and in production use LQCD analysis software suite, speeds up the calculations of LQCD correlation functions through novel algorithmic and software engineering methods. Specifically, first, we illustrate how to use a graph to represent the terms of the *Wick contractions* of a correlation function. Second, we show how to identify unique graphs among all the graphs. Third, we present a naive and an optimal graph contraction algorithms to demonstrate how the optimal algorithm achieves the desired optimizations described in the previous paragraph. Furthermore, we present *Hadron*

contraction library, which accelerates tensor contractions on many-core CPUs and GPUs. Finally, we show the performance figures of *Redstar* for calculating correlation functions of six hadronic systems on CPU and GPU computing platforms to demonstrate that *Redstar* suite indeed is highly efficient for calculating correlation functions in LQCD.

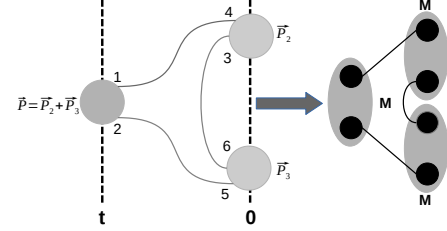


Figure 1: A quark propagation diagram is represented by a graph where $\vec{p}$, $\vec{p}_2$ and $\vec{p}_3$ are the momenta of the hadrons at time slice t and 0 .

2 GRAPH CONTRACTIONS

A natural way to represent the sum of *Wick contractions* is a set of quark propagation diagrams describing quark contractions such as $S_{i,j}$ at a specific time interval. *Distillation* [18] provides an efficient computational algorithm to carry out these contractions. Distillation is a quark smearing method [1]. Treating smearing labels as indices, an example of a quark propagation diagram for one particular combination of two source momenta is shown on the left of Figure 1. On the right of Figure 1, the left quark propagation diagram can be represented as a graph, where a solid circle denotes a quark and an edge connecting two solid circles designates a quark propagation while the **hadron** nodes, each of which contains multiple quarks and is shown as an ellipse, corresponds to the pairs of sink-source terms.

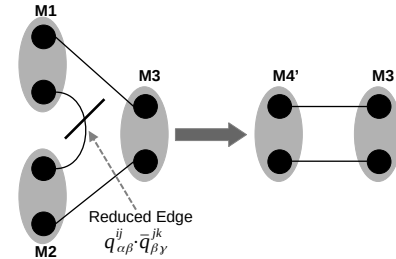


Figure 2: An edge reduction for a meson system. A single edge removal is a batched tensor contraction.

Specifically, a graph contains a set of hadron nodes each has vertices (V) representing the quarks inside the hadron node and undirected edges (E) describing quark propagation at a specific time interval. A single quark can be written as a tensor $q_{\alpha\beta\gamma}^{ijk}$ involving both distillation and spin indices. Therefore, a quark contraction is a batched tensor contraction composed of a set of independent tensor contractions (see Figure 10 for detailed explanation) and can be considered as an elimination or a contraction of the corresponding edge in the graph. Thus, the order of the calculation of a single term of *Wick contractions* can be described as a **graph contraction** which deletes one edge after another until two connected hadron nodes left. Figure 2 shows a single step of a graph contraction for

a simple graph, where M1, M2, and M3 are hadron nodes and M4' is a new hadron node formed by combining two original hadron nodes by deleting an edge (**edge reduction**). The right part of the graph is the final state of the graph contraction with only a pair of hadron nodes left. More importantly, many simultaneous graph contractions can generate the order of the calculation for every single term of the *Wick contractions*.

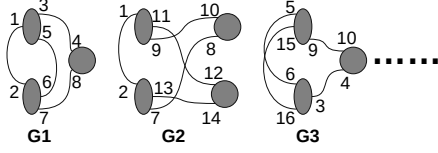


Figure 3: The same edges can appear in different graphs.

In order to obtain physics observables, it is necessary to calculate correlation functions for all time slices, t , followed by averaging the functions for fixed time separations over the temporal extent of the lattice. Even for a fixed time slice, there are many contractions represented by many different graphs. Since two different contractions can have the same quark propagators, two different graphs can share the same edges. Since each edge represents a batched tensor contraction, a repeated edge in multiple graphs presents an opportunity to reduce the duplicated calculations for the same edge by caching the result of the first appearance of the edge. Figure 3 illustrates an example that multiple graphs can have the same edges, where ellipses and circles are baryons and mesons respectively.

3 GRAPH CLASSIFICATION

All the contraction graphs, corresponding to those terms in *Wick contractions*, look different from each other. However, one graph can be the same as another if the hadron nodes in the graph are permuted without violating the following rules because of the nature of physics: 1) permutations can be only done at hadron node level; 2) the order of the vertices, which are shown as small circles inside the ellipses in the example graphs shown in Figure 4, can not be changed; 3) the hadrons sitting on source t cannot change their positions with the hadrons at destination t' . More importantly, the number of graphs is proportional to $N!$, where N corresponds to the total number of quark degrees of freedom at a time slice. Hence, it is vital to find out unique graphs among all the graphs to reduce the number of contractions.

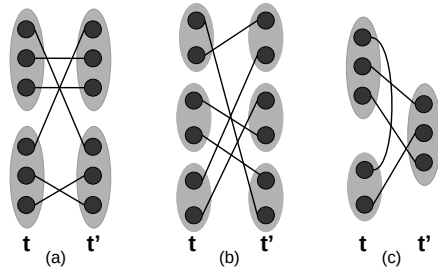


Figure 4: Example contraction graphs, where small solid circles represent quarks, large ellipses serve as hadron nodes, and t and t' are the source and sink of quark propagation. Figure (a), (b), (c) show contraction graphs for nucleon-nucleon, three-pions and nucleon-pion, respectively.

The simplest way to identify the unique graphs within a set of graphs is a straightforward $O(n^2)$ algorithm, where n is the number of graphs. First, for every unclassified graph in the set, this simple algorithm generates many graphs through the eligible permutations of the nodes. Then the algorithm attempts to match one of the generated graphs to one of the already identified unique graphs. When the number of graphs is not huge and the complexity of the graphs from contractions is relative simple, the performance of the algorithm is tolerable. As the number of graphs increases and the complexity of the graphs escalates because of more baryon contractions, the time to classify all the graphs become a significant part of *Redstar* workflow. Therefore, a new classification algorithm is clearly needed.

A **graph isomorphism** [14] of two graphs is a bijection between their vertex sets that preserves adjacency. The graph isomorphism problem (GI) is to determine whether there is an isomorphism between two given graphs, i.e., the two graphs are topologically identical. The most common practical approach is "**canonical labeling**" [27], a process in which a graph is relabeled in such a way that isomorphic graphs are identical after labeling. The relabeled graphs can be easily classified by their associated hash values through a hash table which yields $O(n)$ complexity. In addition, the process of canonical labeling for all graphs can be carried out in parallel on any modern computing platform to further increase the performance of the classification process.

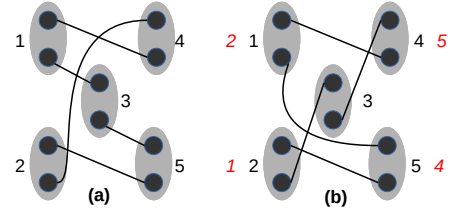


Figure 5: An example of graph isomorphism through graph relabeling.

In *Redstar*, we adopt a well known graph isomorphism software package called *nauty* [14] which has a set of procedures for testing graphs for isomorphism and especially can perform canonical labeling. Graphs that are isomorphic (the same except for vertex labels) become identical after canonical labeling. For example, contraction graph (a) and (b) in Figure 5 are isomorphic which can be easily verified by simultaneously swapping the labels of two pairs of nodes in the graph (b): nodes 1 and 2 as well as nodes 4 and 5.

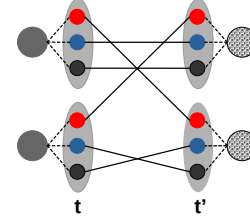


Figure 6: Derived graph with added combo-vertices shown in larger filled circles and colored quark vertices for *nauty* canonical labeling process.

During a canonical relabeling process, the vertices in a graph are remapped but the connections among the vertices are kept intact.

However, a vertex which represents a quark, in a contraction graph can be only mapped to another quark that has the same feature. In addition, all the quarks inside a hadron have to be mapped together. Fortunately, *nauty* supports a notion called *colored vertices* and it respects colors: each vertex can only be mapped onto a vertex of the same color. To ensure that all the vertices inside a hadron are mapped together to another hadron, a new combo-vertex is created for each hadron node. Figure 6 shows one example graph derived from its associated contraction graph for a *nauty* canonical labeling process.

To identify unique graphs using graph isomorphism, *Redstar* utilizes canonical labeling for each graph in parallel, using a simple OPENMP loop to process all the graphs. Because the number of vertices, which is the number of quarks under study, does not exceed 64, the high performant *nauty* graph data structure represented by a long integer bitmask is selected. The output of the canonical labeling of a graph is an integer hash value, which is inserted into a hash table. The process of identifying the unique graphs of a large list of given contraction graphs can be easily done by comparing the hash value of the canonical graph generated for each contraction graph to the entries in the hash table, and the process clearly completes in $O(n)$ steps.

4 CONTRACTION ALGORITHMS

A graph contraction on an undirected graph can be carried out in many different ways since there are more than one edge to chose at any given step. A tree graph can be generated if each and every possible path of the graph contraction is recorded. The number of paths from the tree root node, which is the state of the initial unreduced graph, to the final state is simply $\frac{n!}{2}$, where n denotes the number of edges in the unreduced graph. The number of nodes of the generated tree graph is $\sum_{i=2}^{n-1} \frac{n!}{i!}$, where each node represents a state of the contraction. Moreover, each edge of the tree graph records a single graph edge reduction, which contains all the information about the corresponding matrix/tensor contraction with some computing cost. Since for every contraction graph, there is a generated contraction path tree, the total number of contraction path trees is the same as the number of contraction graphs.

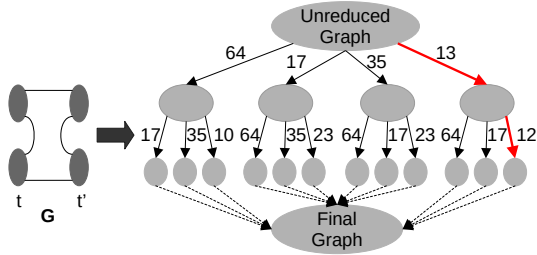


Figure 7: An example contraction path tree generated for a 4-edged graph. The path in red is the selected path.

Figure 7 shows an example of the generated contraction path tree for a 4-edged graph. The dashed lines are added to connect the final state of the graph contraction to the tree for clarity. Each path from the root node of the tree to the final state node represents a particular order of executing one term of *Wick contractions*. Because each tree edge has a computing cost, i.e. weight, associated with it, it is ideal to select a path that incurs the least overall cost. There

are two factors contributing to the computing cost associated with an edge: the first is the number of times this edge appears in all the graphs, which measures the re-usability of this edge; the second is the complexity of the tensor contractions represented by this edge, which can be quantified as $O(N^3)$ for mesons and $O(N^4)$ for baryons, where N is the size of the tensors. The number next to each tree edge in Figure 7 is the weight, which is defined later in this section, of the corresponding reduced graph edge. Moreover, the path in red shown in the figure is the selected contraction path for the example graph.

To reduce the computing cost, we need not only pick a contraction path that has the highest total re-usability from the comprised edges among all possible paths, but also select contractions with less computing complexity and smaller memory footprint for the transient contraction values. Especially, for a hadronic system consisting of both mesons and baryons, *Redstar* prefers contractions between a baryon and a meson and avoids 1-index contractions between two baryons. The latter case requires $O(N^5)$ computing complexity and results a tetra-quark system with memory footprint in the order of $O(N^4)$. Figure 8 illustrates the importance of selection of an optimal contraction path.

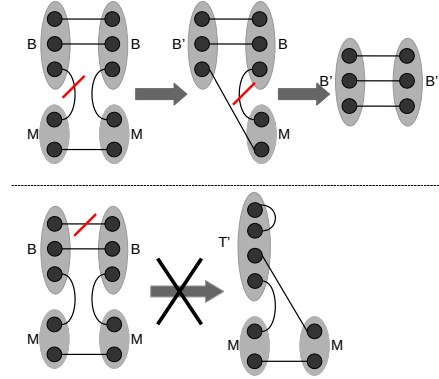


Figure 8: The top graph shows a good contraction path selection. The bottom graph illustrates a non-optimal contraction path selection which results a tetra-quark system.

A simple **naive approach** to contract a graph is to eliminate the edges, which are stored in an adjacency matrix, sequentially until there are only two hadron nodes left. In comparison, our **optimal graph contraction algorithm** comprises three components: a searching algorithm that discovers the number of times an edge appears in all the graphs; application of Dijkstra's [21] algorithm that determines the shortest path from the root node of every tree graph to the final tree node; sorting of all the graphs that groups similar graphs together.

The edges E of all the graphs are visited simultaneously by multiple threads so that the number of times of an edge appeared in all the graphs, which is denoted as E_N , is tagged to every single edge. The total run-time complexity of the assignment is simply $O(\frac{|E|}{\text{Number of Threads}})$. Since an edge E represents a batched tensor contraction, the computing complexity for the edge is assigned using a positive number $C = N^p$, where N is the size of the tensors and p can be 3, 4, and 5 depending on the type of the batched tensor

contractions. A cost function, which is defined as $\frac{C}{E_N}$, is finally assigned as the **weight** for every edge across all the graphs.

After the weight is assigned to every edge, a contraction path tree is generated for each graph. Dijkstra's algorithm is applied to every tree graph to find the shortest path, i.e. smallest weight, from the root node of a graph to the final state tree node. For a multi-meson system, which has the same computing complexity for all the edges, the shortest path of a tree graph has the largest total E_N which yields the highest total computing re-usability and leads to the least total number of contractions for all the graphs. For a hadronic system involving baryons, the shortest path of a tree graph guarantees the least computing cost by eliminating some of the most expensive calculations such as baryon's 1-index contractions and by promoting re-usability of those contractions with the high number of appearances.

Since a tree graph is a direct acyclic graph (DAG), the complexity of running Dijkstra's algorithm is $O(|E| + |V|)$ [21] which is faster than $O(|E| + |V|\log|V|)$ for a general undirected graph. The selected path records the order of the contraction calculations of a graph and convert the order into a task tree. Furthermore, all the task trees, which share some common nodes and edges, are combined into a large contraction **evaluation DAG** for all the graphs. This large DAG not only determines the order of contractions but also decides when a contraction object can be deleted once the object is no longer used by any graph.

During the calculations of contractions, the result of an edge reduction, which is a temporary hadron object, will be cached and will not be deleted until the last graph that uses the cached object is done with it. The memory usage of these cached objects grows rapidly when the number and the complexity of graphs increase. The maximum number of stored objects in memory when evaluating all the graphs is denoted as the High-Water-Mark (**HWM**). Due to the limited memory size on a given computing host and GPUs, a large **HWM** may prevent the calculations of correlation functions for a large physics problem from being carried out. To reduce the value of **HWM**, the graphs are sorted using their string representations before the optimal graph contraction algorithm is applied. This makes the similar graphs clustered together when the large evaluation DAG is constructed. Therefore, the cached result of a contraction can be deleted sooner since the last graph using this result shows up sooner in the newly reordered calculations.

5 MAJOR SOFTWARE COMPONENTS

The correlation functions are constructed and evaluated using the *Redstar* package and *Hadron* library. The latest release of the software packages are available on GitHub at <https://github.com/JeffersonLab/redstar> and <https://github.com/JeffersonLab/hadron> for *Redstar* and *Hadron* respectively.

5.1 Redstar

The *Redstar* code is effectively a workflow engine that coordinates several stages of calculations using the Distillation method [18]. The schematic structure is shown in Figure 9. First, it takes a physics input describing a correlation function through a large XML file, symbolically constructs *Wick contractions*, determines the topologically unique graphs that must be evaluated, and outputs the graphs

with associated hadron nodes. Second, for the first time slice, it applies the optimal graph contraction algorithm to generate a large evaluation DAG which describes the contraction order and subsequently is converted to a contraction queue. In addition, *Redstar* groups a set of independent contractions together when possible in order to carry out the set of contractions in parallel. In every time slice, *Redstar* creates all required hadron nodes and passes the contraction queue along with the hadron nodes to *Hadron* library which carries out tensor contractions on CPUs or GPUs to generate an array of complex numbers for this time slice. Finally, *Redstar* assembles those arrays of complex numbers to obtain the result for the correlation function.

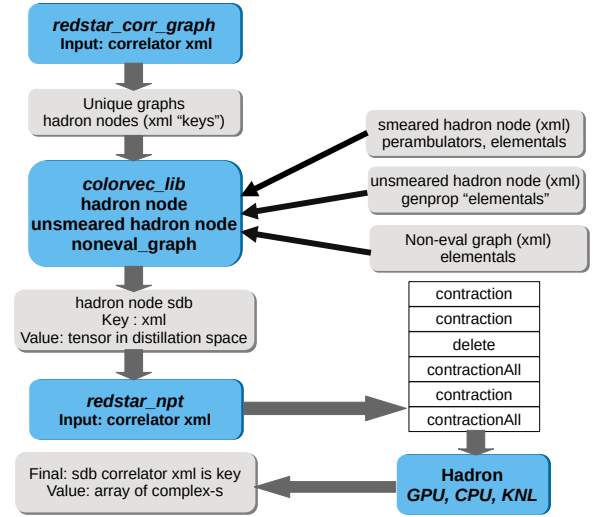


Figure 9: Schematic structure of the *Redstar* package.

5.2 Hadron Library

A single hadron node is defined in *Hadron* library as a C++ object which describes a physics hadron object of $h_{ijk}^{\alpha\beta\gamma}$, where $\alpha, \beta, \gamma \in [0, 4)$ are spin indices and i, j, k stand for distillation space indices with values denoted by N , where $N \approx 100$ s for multi-baryon systems and $100 < N < 1000$ for multi-meson systems. Therefore, a single hadron object contains many spin components, each of which is a dense tensor of double complex of size N with rank 2 and 3 for mesons and baryons respectively. A one index contraction is expressed using Einstein notation as $a_{ijk}^{\alpha\beta\gamma\delta} = b_{ijl}^{\alpha\beta\lambda} \cdot c_{lkm}^{\lambda\gamma\delta}$. Similarly, a two-index contraction can be written as $a_{ij}^{\alpha\beta} = b_{ikl}^{\alpha\delta\lambda} \cdot c_{klj}^{\delta\lambda\beta}$. Thus, a hadron contraction can be seen as a sum of tensor contractions on the distillation space with the number of terms determined by the same contraction on spin indices. Figure 10 describes graphically how a hadron contraction is turned into a batched tensor contraction. The final contraction result for a particular spin is obtained by accumulating the tensor contraction values contributing to this spin. In addition, tensor permutations may be necessary for the tensors in a contraction if the indices of the tensors are not in appropriate positions. Finally, the operation of *contractionAll* is simply defined by the expression $(a_{ijk}^{\alpha\beta\gamma} \cdot b_{ijk}^{\alpha\beta\gamma})$.

It is difficult to achieve high performance for tensor contractions on regular CPUs and GPUs [22]. For hadron contractions, there are

additional and unique challenges. First of all, a hadron object has a substantial memory footprint and is internally represented as a set of spatial dense tensors keyed by the spin indices. For example, a single meson with 16 spin components and the dimension size 384 occupies 37 MB, a single baryon with 64 spin components and the dimension size of 128 takes up 2 GB. In a typical *Redstar* calculation, there is a large number of hadron objects including those intermediate hadron objects produced by the contractions, which results a very large memory footprint and generates immense data transfers between a host and its GPUs with potential more evictions of GPU resident data due to the limited memory size of a single GPU. Secondly, a single tensor contraction may not provide good performance because the size of the tensors under consideration is not large enough to fully utilize all computing cores on a CPU or a GPU. Thirdly, a naive tensor permutation on a CPU and a GPU delivers abysmal performance because of its strided memory access. Finally, some matrix multiplications during tensor contractions become matrix-vector (*gemv*) or dot-product (*udot*) because of unfolding of the tensors, which lead to sub-optimal performance. To deliver high performance for hadron contractions on GPUs, these challenging issues have to be addressed.

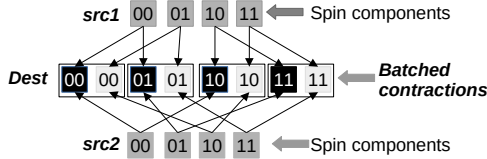


Figure 10: Graphical illustration of a hadron contraction resulting a batched tensor contraction from $dest=hadronExteriorContraction(src1, 1, src2, 0)$. For illustration purpose, $src1$, $src2$ and $dest$ have spin size of 2. Each source spin component contributes to two destination spin components. The batch size of the batched tensor contraction in this case is 8. The intermediate values contributing to the same spins for the $dest$ are merged together.

Our *Hadron* library defines a set of standard C++ APIs through some C++ virtual base classes to handle various flavors of contractions. These APIs are implemented to target computing platforms such as simple CPU, Intel KNL [23] and GPUs to address the challenges described in the previous paragraph using a set of C++ classes inherited from the C++ virtual base classes.

On a many-core CPU system such as a KNL host, *Hadron* packs a set of independent hadron contractions generated by *Redstar* together to execute "a group of batched tensor contractions" provided by MKL [11] in order to fully utilize the computing power offered by all computing cores. In addition, a fast tensor permutation library *mctt* based on the tensor transpose library inside *TAL_SH* [13] package is implemented to further enhance the performance of tensor permutations for a group of tensors simultaneously.

To take advantage of the processing power of Nvidia and AMD GPUs, *Hadron* library implements the necessary C++ derived classes utilizing CUDA [16] toolkit and ROCm [2] framework. It undergoes many optimizations [25, 26] to provide the best possible contraction performance on GPUs. The followings are some of its features and optimizations:

- On every GPU, a GPU memory manager manages the GPU memory of hadron objects to reduce data allocations and movements without resorting to the Unified Memory management. It uses a modified LRU [17] GPU memory eviction policy to allow over-subscription of GPU memory.
- A contiguous memory layout for all the spin components of a hadron object reduces the number of memory movements between a host and GPUs.
- A fast tensor permutation library, *gputt*, is developed based on *cutt* [10] to further improve the performance of batched tensor permutations for Nvidia and AMD GPUs.
- A highly performant on-line data placement and task scheduling algorithm for multiple GPUs to reduce data movements and maintain a good load-balance among GPUs.

6 TEST SETTINGS

In this paper, we present the performance results of calculating several correlation functions with the basis size N of distillation space using randomly generated hadron nodes. In order to achieve a good statistical accuracy, it usually requires $N = 256$ and four time sources for the correlation functions. However, here we only need to consider just two time slices in the correlation functions, one-time source and only one configuration. In addition, for the nucleon systems, we cannot evaluate the correlation functions at $N = 256$ because of the memory limitation of the testing hosts, thus we will use a reference size N_{ref} . Table 1 shows the information about the correlation functions along with complexity of the calculations, where M and B denote mesons and baryons, respectively, and N is the size of the matrices and tensors.

#	Name	Type	# graphs	N_{ref}	Complexity
1	a_0	$M \times M$	19041	256	$O(N^3)$
2	f_0	$M \times M \times M$	29600	256	$O(N^3)$
3	a_1	$M \times M \times M$	403072	128	$O(N^3)$
4	Roper	$B \times M$	84894	64	$O(N^4)$
5	Deuterium	$B \times B$	119191	64	$O(N^4)$
6	Tritium	$B \times B \times B$	6208	32	$O(N^5)$

Table 1: The correlation functions under study.

We evaluate the correlation functions on four different platforms: (1) Intel® KNL system with a Xeon Phi 7230 CPU, which contains 64 cores running at 1.3 GHz and 192 GB memory; (2) Nvidia® A100 GPU system with 4 A100 GPUs. Each GPU has 40 GB on board GDDR5 memory and is connected to the other GPUs and the host via NVLink and PCI-4 bus respectively. The host has one AMD EPYC 7763 CPU consisting of 64 cores running at 2.45GHz and 256 GB DDR4 memory; (3) AMD® MI100 GPU system with 8 MI100 GPUs. Each GPU has 32 GB on board GDDR5 memory and is connected to the other GPUs and the host through Infinity Fabric and PCI-4 bus respectively. The host has 1 TB DDR5 memory and two AMD EPYC 7502 CPUs each having 32 cores running at 2.5 GHz; (4) AMD® MI250 GPU system with 4 MI250X containing 8 separate MI250 GPUs. Each GPU has 64 GB on board GDDR5 memory and is connected to the host and the other GPUs via Infinity Fabric. The host has one AMD EPYC 7A53 CPU having 64 cores with 512 GB DDR5 memory. On the KNL system, Intel parallel studio version 19, which contains Intel MKL system, is used to compile and execute

Redstar and other benchmark programs. On the *A100* system, *CUDA* toolkit version 11.7 and *gcc* version 11.2.0 are utilized. Finally, on the *MI100* and *MI250* systems, *ROCM* framework of version 5.1.1 and *gcc* 8.5 are our preferred choices.

7 PERFORMANCE RESULTS

Graph classifications and graph contraction algorithms are carried out on CPUs regardless of the presence of GPUs. Therefore, the performance results of the graph classifications and the graph contractions algorithms are collected on the *MI100* system using 64 hardware cores.

7.1 Graph Classification

We collect the total time of generating all unique contraction graphs for the correlation functions listed in Table 1 using the algorithm taking advantage of parallel canonical labeling of graphs and the simple $O(N^2)$ naive method. Figure 11 presents the speedup values.

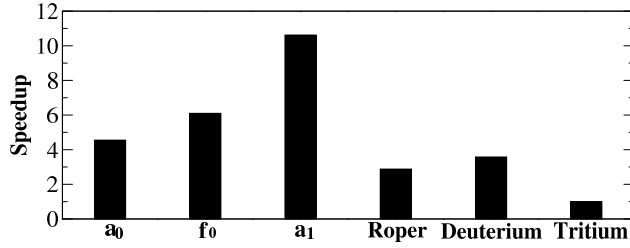


Figure 11: The speedup values of using new graph classification algorithm over the naive method.

It is clear that the new graph classification algorithm through *nauty* canonical labeling indeed provides good speedup values throughout the test correlation functions. The speedup values range from 1.01 to 10.62. The performance improvement of graph classifications depends on several factors such as the number of graphs, the number of duplicated graphs, and complexity of the graphs.

7.2 Graph Contraction Algorithms

To accurately reflect the benefit of our optimal graph contraction algorithm in comparison to the naive approach, we calculate the correlation functions listed in Table 1 using *Redstar* and count the number of unique edge contractions along with the *HWM* values.

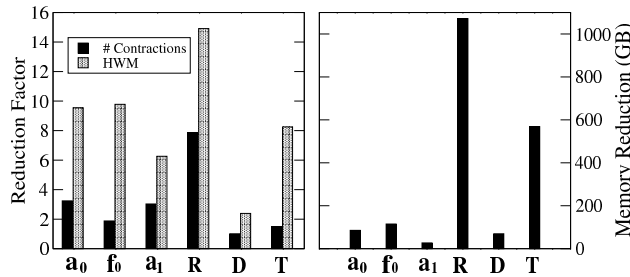


Figure 12: The left figure shows the reduction in the number of contractions and *HWM*. The right figure displays the saved memory size. R, D, and T represent Roper, Deuterium and Tritium respectively.

From the left side in Figure 12, it clearly shows that the number of unique edge reductions/contractions of the optimal algorithm

decreases compared to that of the naive approach across all the correlation functions. The factor of the decreases is ranging from 1.01 for *Deuterium* to a large 7.86 for *Roper*. This illustrates the fact that the optimal algorithm indeed takes full advantage of the reusability of edge contractions to reduce the number of calculations for the correlation functions.

Furthermore, as shown on the left side of Figure 12, using the optimal algorithm helps to lower the number of stored contraction values or *HWM* by 2.39 to 14.91 times for all the functions in comparison to the naive approach. This directly reduces the total memory usage, and the right side of Figure 12 presents the saved memory size in GB for the calculations. In particular, *Redstar* reduces more than 1 TB memory for *Roper* system.

7.3 Hadron Contractions

Hadron library carries out the contractions on behalf of *Redstar* which generates a contraction queue using the optimal graph contraction algorithm. The contraction queue can contain either many individual contractions or arrays of independent contractions. *Hadron* library can execute the contractions either on a many-core CPU or on GPUs utilizing batched *zgemms*. Several important performance optimizations have been applied to *Hadron* library, especially for the GPUs as described thoroughly in [25, 26]. Figure 13 compares the performance between batched *zgemms* from vendor provided math libraries, such as MKL and cublas/hipblas, and hadron contractions from *Hadron* library.

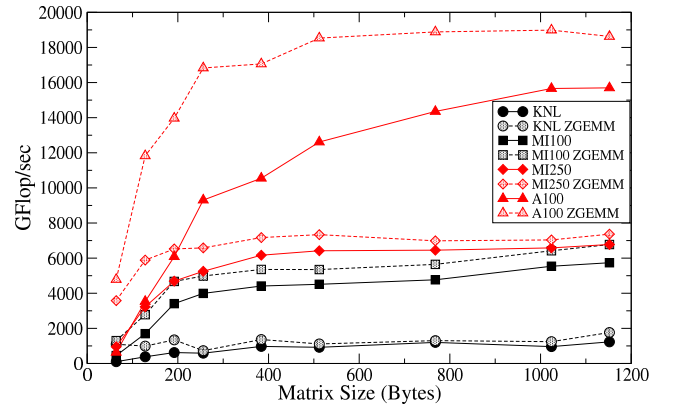


Figure 13: The performance comparison between batched *zgemms* and hadron contractions on a KNL host and various single GPU.

In Figure 13, the batch size of the batched *zgemms* is 64 because every hadron contraction test uses the batch 64 for the internal batched tensor contraction to simulate the contractions of a multi-meson system when the complete set of spins is used. Each hadron contraction performance test fits in the single GPU without generating any memory eviction. The performance values of the hadron contractions are around 75% of the corresponding batched *zgemms* for matrix size above 256. However, for the smaller matrix size, the performance of hadron contractions does not fare well in comparison to the pure batched *zgemms* because of the overhead of bookkeeping for contractions in *Hadron* library. Fortunately, for typical LQCD applications, the region of matrix size between 256 and 800 is the area of our interest. Figure 13 shows that *Hadron*

library provides highly efficient implementation of hadron contractions especially on GPUs. More interestingly, in terms of the performance of hadron contraction and batched *zgemv*, one A100 offers a little more than twice as much what one MI250 can provide and has about 11 times as much as one KNL host can deliver.

To take advantage of the computing power provided by multiple GPUs on a single host, many scientific applications utilize the available GPUs in order to achieve better performance. However, the majority of the applications have data communications among the GPUs, which impacts the overall performance [12]. The data communications can be realized through several methods either by IPC coupled with NVSHMEM/ROC_SHMEM [15, 19] or by *cuda/hipMemcpyAsync* using multiple threads each managing a particular GPU. *Hadron* library takes the latter approach to move data from one GPU to another asynchronously when the same datum is needed by multiple GPUs or to achieve better data locality and load balance [26].

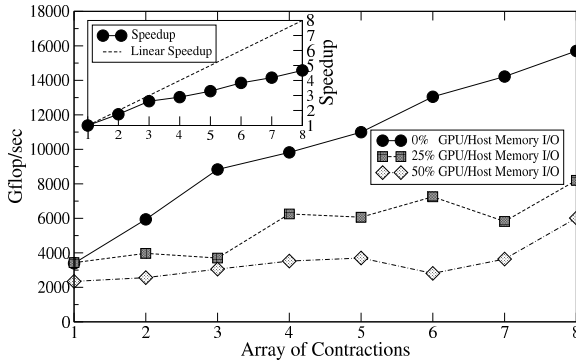


Figure 14: The performance of hadron contractions on multiple MI100 GPUs with matrix size of 384.

Figure 14 shows the performance data from three different runs of a synthetic hadron contraction experiment using one to eight MI100 GPUs on a single host. During the course of a run, we execute many loops of an array of contractions with the size of the array the same as the number of used GPUs, each GPU gets one contraction from two hadron objects in every loop. However, for a particular run, we copy a certain percentage of hadron objects from host and keep the rest in the GPU memory in every loop. Each contraction pair in an array is formed randomly, which may lead to a source data being used in several contractions of the same array. Figure 14 contains two plots: one large plot shows the performance of hadron contractions on multiple GPUs for three different percentages of host-to-GPU data source transfers; the small plot shows the speedup values of using multiple GPUs when there is no host-to-GPU data transfers. Thus, Figure 14 illustrates the impact to the performance due to two types of data movements: one is between GPUs and host, and the other is between two different GPUs. First of all, it is clear that the overall performance rises as the number of used GPUs increases. But, the tests with larger host-to-GPU memory transfers produces inferior contraction performance in comparison to the tests with smaller host-to-GPU memory data movements. Secondly, the speedup in the case of zero host-to-GPU memory transfers is smaller than a linear speedup, which reveals the true impact to the performance due to the data transfers among GPUs.

7.4 Redstar

Redstar utilizes *Hadron* library to carry out hadron contractions after it generates a contraction queue using the optimal graph contraction algorithm. The contraction queue is constructed once for all the time slices and each element of the contraction queue contains all necessary contraction information such as hadron source elements and the contraction indices. However, the source elements inside the queue are meta hadron nodes which contain all the information of the nodes without real values. The meta hadron nodes are loaded with correct values when they are being used for a specific time slice throughout all the time slices.

To understand the intrinsics of its performance, *Redstar* records every floating point operation (**flop**), the number of bytes transferred, and the number of evictions in addition to logging the timing information of several important execution steps for each and every time slice. In this paper, we show the above information for the first time slice when *Redstar* calculates the six correlation functions defined in Table 1 on the testing platforms with up to 4 GPUs.

Name	Classification	DAG	Contraction	Time
a_0	12.5	19.1	10.5	42.1
f_0	28.5	31.9	16.6	77.0
a_1	925.5	929.4	117.1	1972
<i>Roper</i>	37.2	137.9	1924.1	2099.2
<i>Deuterium</i>	122.4	191.7	501.4	811.5
<i>Tritium</i>	37.1	19.3	2952.3	3008.7

Table 2: *Redstar* execution timing information (second) for the first time slice on a MI100 host using a single GPU.

Table 2 shows the detailed timing information of *Redstar* in the unit of seconds during its calculations of the correlation functions for the first time slice on a single MI100 host using a single GPU. The rightmost column presents the total execution time for each calculation. The Classification, DAG and Contraction columns show the running time for graph classification, construction of the DAG including the preparation of graph contraction path tree, and hadron contractions on the MI100 GPU, respectively. It is clear that the graph classification and DAG construction occupy some significant fractions of the total execution time for the meson systems, especially a_1 which has a very large number of graphs with smallish tensors. In contrast, the contraction time is the dominant factor of the total time for the baryon systems because of the high cost of tensor contractions and the smaller number of graphs. More importantly, graph classification and DAG construction for a correlation function are only necessary for the first time slice using the very first gauge configuration during a campaign run on many configurations. Therefore, the time spent on these critical two steps is amortized over multiple time slices and many configurations.

Figure 15 shows the effective performance of *Redstar* for the first time slice in the unit of *GFlop/sec* for calculating the correlation functions using one GPU on the three GPU platforms in addition to running on one KNL host. For a single KNL host, using 64 OPENMP threads, each running on a core, *Redstar* achieves the performance from 59 *GFlop/sec* to 936 *GFlop/sec*. For the three GPU systems, A100 has the best performance for all the correlation functions except *Roper*. This is expected because of the better performance

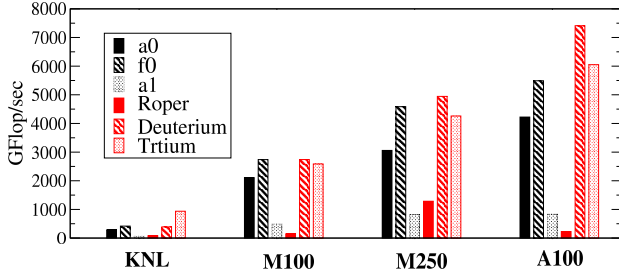


Figure 15: The performance of *Redstar* of a single time slice for calculating the correlation functions using a single GPU.

of hadron contractions and batched *zgemms* on a single A100 in comparison to the performance on a GPU of the other two types. In particular, on A100 system, *Redstar* delivers the performance ranging from 228 GFlop/sec to 7410 GFlop/sec, which is roughly the half of what batched *zgemms* provide. In particular, for the three multi-baryon systems, there are many GPU memory evictions. However, *Deuterium* and *Tritium* achieve high performance partially due to the high flop counts because of the rank 3 tensors coupled with the large batch size of the tensor contractions. Unfortunately, *Roper*, which has meson and baryon contractions, attains not impressive performance because of its small matrix size of the mesons and its compact batch size of the tensor contractions. In contrast, for the three multi-meson systems, there are no GPU memory evictions and all the intermediate results stay in a GPU. Therefore, *a0* and *f0* obtain very good performance meanwhile *a1* manages rather poor performance due to the small matrix size of 128.

Name	# GPUs	Evictions	Bytes(GB)	Flops/Byte
Roper	1	32356	16074	17
	2	15920	10729	26
	4	1671	6302	45
Deuterium	1	17767	9538	148
	2	1340	9159	154
	4	667	9013	157
Tritium	1	2749	6959	1104
	2	2671	4807	1598
	4	2354	4039	1903

Table 3: Number of evictions and the total number of bytes transferred for the three multi-baryon systems in one time slice on multiple MI100 GPUs.

Many scientific applications take advantage of multiple GPUs on a single host in order to achieve better performance. As illustrated in Figure 14, *Hadron* library has the capability of delivering performance speedups when hadron contractions are executed on multiple GPUs. Unfortunately, for the three multi-meson systems, there are no memory evictions and all the intermediate results reside in the GPU when *Redstar* runs on a single GPU. Therefore, it is not beneficial for *Redstar* to utilize multiple GPUs to compute the correlation functions for the three multi-meson systems.

For the three multi-baryon systems, there are many GPU memory evictions during the calculations of the correlation functions on a single GPU. Utilizing multiple GPUs offers more GPU memory which benefits those applications. However, the data movements

among multiple GPUs from an application may offset the benefit from the reduction of GPU memory evictions, which could lead to a mediocre performance increase for the application. Table 3 shows the number of evictions and the total number of bytes transferred, which includes data moved between host and GPUs as well as data transferred among GPUs, in one time slice when *Redstar* executes on one, two and four MI100 GPUs for the three applications. As the number of GPUs increases, which increases the total amount of GPU memory, the number of eviction drops while the number of data transfers among the GPUs escalates. However, the total combined data movements actually decrease coupled with the increase of the performance. Thus, the performance on multiple GPUs depends on how to reduce the data movements. In addition, the last column of Table 3 shows the ratio of the number of flops to the number of bytes transferred. This ratio reflects how data transfers impact to the performance of the applications running on multiple GPUs.

Moreover, the right plot in Figure 16 shows the *Redstar* performance for the three multi-baryon systems using one, two and four MI100 GPUs. *Tritium* system has the best performance increase when it uses multiple GPUs due to its smaller number of data evictions and bytes moved. Unfortunately, the other two systems obtain only moderate performance increase because of their large number of bytes transferred when using multiple GPUs. In particular, the performance increases are directly related to the increase of the ratio of flops to bytes transferred shown in Table 3.

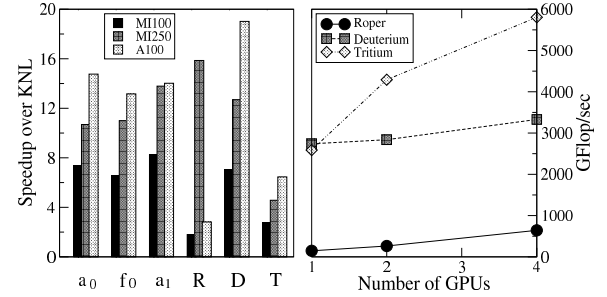


Figure 16: The left plot shows the performance speedup for one GPU over a single KNL host, where R, D, and T stand for *Roper*, *Deuterium*, and *Tritium*, respectively. The right plot illustrates the performance of multi-baryon systems for a single time slice when calculating the correlation functions on multiple MI100 GPUs.

Finally, the left plot in Figure 16 highlights the performance speedup of *Redstar* running on a single GPU over performing on one KNL host for the correlation functions. It is clear that all three type of GPUs achieve good speedup values with A100 having the best performance boost. Specifically, the speedup values for one A100 GPU ranges from 2.8 to 19 with an average speedup value of 11.7 which is very much inline with the performance ratio of hadron contractions running on a single A100 to performing on one KNL host. This showcases that the optimizations invested in GPU implementation of *Redstar* are indeed highly effective.

8 RELATED WORK

Several algorithmic and software advancements have been made in the evaluation of LQCD correlation functions in recent years. There are two notable works: one is sLapH-contractions [7]. It

is a C++ framework using a stochastic variant of the Distillation method [6] to estimate hadron correlation functions. In this approach, distinct stochastic noise sources are used for the computation of the propagator factors. For nuclei, the cost of computing these propagators grows factorially with the number of quark fields and stochastic noise source permutations; The other work is the Contraction Optimizer [24]. It is another C++ library utilizing Common-Subexpression Elimination and Operation Minimization algorithms [8, 9] to reduce the number of contractions and the cost of the contractions for a single term of *Wick contractions*. Unfortunately, it has a small limit on the number of tensors inside a single term of *Wick contractions* and forces the same number of dimensions for all tensors. In contrast, our work, *Redstar*, makes the following unique contributions: using a graph to represent a single term of *Wick contractions*; utilizing graph algorithms to generate a tensor evaluation DAG that reduces the total computing cost; minimizing run-time memory footprint; an integrated tensor contraction engine, *Hadron* library that handles batched tensor contractions efficiently on multi-core CPU and on GPUs with the ability of oversubscription of GPU memory.

9 CONCLUSION AND FUTURE WORK

In this paper, we present a novel method that converts symbolic calculations of many-particle correlation functions, during the stage of *Wick contractions*, into a format suitable to perform optimizations utilizing graph algorithms. In particular, *Redstar* exploits several graph algorithms to find optimal contraction paths for all graphs to reduce the number of contractions and the runtime memory footprint. These optimizations reduce the number of contraction by 1.01 to 7.86 and brings down the HWM values by 2.39 to 14.91. Furthermore, *Redstar* constructs a contraction queue, which consists of arrays of contractions each of which is independent from the others, and directs all the contractions to a highly optimized *Hadron* library to carry out hadron contractions on a many-core CPU or GPUs. The performance of *Redstar* for calculating the six benchmark correlation functions achieves 936 GFlop/s on the KNL system and as high as 7410 GFlop/s on an A100 GPU. Moreover, for the multi-baryon systems, utilizing multiple GPUs indeed provides performance boost for *Redstar* despite the large amount of data movements among the GPUs. To further improve the performance of *Redstar*, we look into applying graph partition technique to divide the large DAG into similar sized partitions with the minimal number of edges crossing the boundaries of the partitions. Each partition will be handled by a dedicated GPU to further reduce the data movements among GPUs.

10 ACKNOWLEDGMENT

This work is partially supported by the US Department Of Energy, Office of Science, Offices of Nuclear Physics and Advanced Scientific Computing Research, through the SciDAC program under contract DE-AC05-06OR23177 under which JSA LLC operates and manages Jefferson Lab, and under the 17-SC-20-SC Exascale Computing Project.

REFERENCES

- [1] C. R. Allton et al. 1993. Gauge-invariant smearing and matrix correlators using Wilson fermions at $\beta = 6.2$. *Phys. Rev. D* 47 (Jun 1993), 5128–5137. Issue 11.

- https://doi.org/10.1103/PhysRevD.47.5128
- [2] AMD. 2022. ROCm - Open Software Platform for GPU Compute. <https://github.com/RadeonOpenCompute/ROCm>
- [3] Christian B. Lang Christof Gattringer. 2010. *Quantum Chromodynamics on the Lattice*. Springer, Berlin, 124. <https://link.springer.com/book/10.1007/978-3-642-01850-3>
- [4] Carleton DeTar and Steven Gottlieb. 2004. Lattice Quantum Chromodynamics Comes of Age. *Physics Today* 57, 2 (2004), 45–51. <https://doi.org/10.1063/1.1688069>
- [5] William Detmold and Kostas Orginos. 2013. Nuclear correlation functions in lattice QCD. *Phys. Rev. D* 87 (Jun 2013), 114512. Issue 11. <https://doi.org/10.1103/PhysRevD.87.114512>
- [6] Justin Foley, Chik Him Wong, John Bulava, K. Juge, David Lenkner, Colin Morningstar, and Michael Peardon. 2010. A novel method for evaluating correlation functions in lattice hadron spectroscopy. (11 2010).
- [7] Lattice QCD group at the University of Bonn. 2019. sLapH-contractions. <https://github.com/HISKP-LQCD/sLapH-contractions>
- [8] Albert Hartono et al. 2005. Automated Operation Minimization of Tensor Contraction Expressions in Electronic Structure Calculations. In *Computational Science – ICCS 2005*. Springer Berlin Heidelberg, Berlin, Heidelberg, 155–164.
- [9] Albert Hartono et al. 2006. Identifying Cost-Effective Common Subexpressions to Reduce Operation Count in Tensor Contraction Evaluations. In *Computational Science – ICCS 2006*. Springer Berlin Heidelberg, Berlin, Heidelberg, 267–275.
- [10] Antti-Pekka Hynninen and Dmitry Lyakh. 2017. cuTT: A High-Performance Tensor Transpose Library for CUDA Compatible GPUs. (05 2017).
- [11] Intel. 2022. Accelerate fast math with Intel® ONEAPI math kernel library. <https://www.intel.com/content/www/us/en/developer/tools/oneapi/oneapi.html#gs.k82vrb>
- [12] Ezhilmathi Krishnasamy, Mohammed Sourouri, and Xing Cai. 2015. Multi-GPU Implementations of Parallel 3D Sweeping Algorithms with Application to Geological Folding. *Procedia Computer Science* 51 (2015), 1494–1503. <https://doi.org/10.1016/j.procs.2015.05.339> International Conference On Computational Science, ICCS 2015.
- [13] Dmitry I. Lyakh. 2015. An efficient tensor transpose algorithm for multicore CPU, Intel Xeon Phi, and NVIDIA Tesla GPU. *Computer Physics Communications* 189 (2015), 84–91. <https://doi.org/10.1016/j.cpc.2014.12.013>
- [14] Brendan D. McKay and Adolfo Piperno. 2014. Practical graph isomorphism, II. *Journal of Symbolic Computation* 60 (2014), 94–112. <https://doi.org/10.1016/j.jsc.2013.09.003>
- [15] NVIDIA. 2022. NVSHMEM: Nvidia. <https://developer.nvidia.com/nvshmem>
- [16] NVIDIA, Péter Vingelmann, and Frank H.P. Fitzek. 2020. CUDA, release: 10.2.89. <https://developer.nvidia.com/cuda-toolkit>
- [17] Elizabeth J. O’Neil, Patrick E. O’Neil, and Gerhard Weikum. 1993. The LRU-K Page Replacement Algorithm for Database Disk Buffering. *SIGMOD Rec.* 22, 2 (jun 1993), 297–306. <https://doi.org/10.1145/170036.170081>
- [18] Michael Peardon, John Bulava, Justin Foley, Colin Morningstar, Jozef Dudek, Robert G. Edwards, Bálint Joó, Huey-Wen Lin, David G. Richards, and Keisuke Jimmy Juge. 2009. Novel quark-field creation operator construction for hadronic physics in lattice QCD. *Phys. Rev. D* 80 (Sep 2009), 054506. Issue 5. <https://doi.org/10.1103/PhysRevD.80.054506>
- [19] ROCm-Developer-Tools. 2022. ROCm-developer-tools/ROC_SHMEM: ROC_SHMEM intra-kernel networking runtime for AMD gpus on the ROCm platform. https://github.com/ROCm-Developer-Tools/ROC_SHMEM
- [20] Matthew D. Schwartz. 2013. *Quantum Field Theory and the Standard Model*. Cambridge University Press, Cambridge, 100–103.
- [21] Robert Sedgewick and Kevin Wayne. 2014. *Algorithms* (4th ed.). Addison-Wesley, Upper Saddle River, NJ. <https://algs4.cs.princeton.edu/home/>
- [22] Yang Shi, U. N. Niranjan, Animashree Anandkumar, and Cris Cecka. 2016. Tensor Contractions with Extended BLAS Kernels on CPU and GPU. In *2016 IEEE 23rd International Conference on High Performance Computing (HiPC)*. 193–202. <https://doi.org/10.1109/HiPC.2016.031>
- [23] Avinash Sodani. 2015. Knights landing (KNL): 2nd Generation Intel® Xeon Phi processor. In *2015 IEEE Hot Chips 27 Symposium (HCS)*. 1–24. <https://doi.org/10.1109/HOTCHIPS.2015.7477467>
- [24] Andre Walker-Loud, Erik Hoerz, and USDOE. 2019. Contraction Optimizer v1.0. <https://doi.org/10.11578/dc.20191015.2>
- [25] Qihan Wang, Zhen Peng, Bin Ren, Jie Chen, and Robert G. Edwards. 2022. MemHC: An Optimized GPU Memory Management Framework for Accelerating Many-Body Correlation. *ACM Trans. Archit. Code Optim.* 19, 2, Article 24 (mar 2022), 26 pages. <https://doi.org/10.1145/3506705>
- [26] Qihan Wang, Bin Ren, Jie Chen, and Robert G. Edwards. 2022. MICCO: An Enhanced Multi-GPU Scheduling Framework for Many-Body Correlation Functions. In *2022 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 135–145. <https://doi.org/10.1109/IPDPS53621.2022.00022>
- [27] Renda Wang et al. 2013. An Efficient Graph Isomorphism Algorithm Based on Canonical Labeling and Its Parallel Implementation on GPU. In *2013 IEEE 10th International Conference on High Performance Computing and Communications*. 1089–1096. <https://doi.org/10.1109/HPCC.and.EUC.2013.154>